Harvey Barnes
Charlie Houchen
10/9/2026

CMPSC 448 Midterm

The data:

For the project we used the lmarena-ai arena human preference dataset on hugging face. It's a public collection of human preference votes on text conversations with LLMs. The original dataset contains 135,634 evaluations involving 53 model names, with recorded timestamps from April 17 through July 24, 2025. Each row compares two models on a users conversation. The model a and model b fields identify the models, conversation a and conversation b contain their conversation histories, and winner records whether the user preferred either model, rated them equally, or thought that both responses were bad. Other fields describe language, whether the conversation involves code, task categories, token, and formatting counts. The dataset provides both generated text and its known source, which makes it useful for our project even though it was originally collected to study human preferences.

For our project, we started by converting each comparison into separate examples containing a model name, user prompt, and model response. We focused on the three largest groups in our mapping, Claude, GPT, and Gemini, rather than going through all 53 individual model names. The GPT group includes OpenAI's o-series, and the Gemini group includes Gemma models.

Preprocessing:

We used the LMArena arena-human-preference-140k dataset (135,634 pairwise battles between anonymous LLMs). We split each “battle” into one row per response and grouped the 53 models into families. For example gpt 3 and gpt 4o would both be under GPT and same goes for models from different companies. We kept the 3 largest which are Claude, Gemini and GPT resulting in a set of 150,735 responses. We kept only single turn conversations with empty response removed, leaving 68,088 responses. Then we dropped exact duplicates, responses under 20 characters. Then we capped each model in a family to 3K responses so no single model represents too much of their family. Then we cut each family to have an equal number of 18,294 responses totalling to 54,882 total.

We split the data 80/10/10 by prompt (43,859 train, 5,486 validation, 5,537 test) so no prompt appears in more than one split. Text was the tokenized with a regular expression

tokenizer that keeps punctuation, capitals and line breaks as separate tokens. A standard lowercasing tokenizer would get rid of formatting information the models could learn from. The 50,000 vocabulary was built from training set only (3.3% of validation tokens fall outside it). Each text was cut or padded to 512 token. Responses were cut at 512 in 19.9% of claude, 58.6% of GPT and 73.8% of Gemini texts. For RQ4 we also built a length controlled set (responses of 256 tokens or longer) and a set with formatting stripped (markdown, emojis, dashes etc).

Step	Rows remaining
Claude, Gemini, GPT responses	150735
Single-turn, non-empty	129263
English	68088
After deduplication and length filter	67666
After per-model cap and balancing	54882

Models and Training:

We implemented the two required models in PyTorch a convolutional network (CNN) and a recurrent network (a bidirectional LSTM).

CNN: Each token is mapped to a 128 dimensional embedding that is learned during training. Three parallel 1-D convolutions with window sizes 3, 4 and 5 (128 filters each) scan the sequence so each detects patterns, like as an opening phrase or markdown symbols. After a ReLU max pooling over the whole response keeps each filters greatest match and the three pooled vectors (384 values) are put together, given through dropout (0.5) and given to a linear layer that outputs one score for each of the three families.

Bidirectional LSTM: The same 128 embeddings feed a single bidirectional LSTM layer with 128 hidden units per direction. The model reads each response both ways (start to finish and other way around) and it uses the true response length so padding is ignored. The final hidden states of the two directions are combined (256 values), passed through dropout (0.5) and given to a linear output layer. We choose an LSTM over a regular RNN because plain RNNs struggle to learn from long sequences (up to 512 tokens).

The LSTM can carry information across the whole response for example from the opening sentence to the closing question.

Training: Both models used cross entropy loss and the Adam optimizer (learning rate 0.001), with batches of 64 and gradient clipping at 1. Training runs at most 10 epochs, and after each epoch we measure accuracy on the validation set. Stop when validation accuracy has not improved for two epochs and keep the weights from the best epoch. The test set is used only once at the end. Because the classes are balanced, we use no class weights. The CNN has about 6.6M parameters and BiLSTM about 6.7M of which 6.4M are the embedding table in both cases. On a GPU an epoch takes about 17 seconds for the CNN and 30 seconds for the BiLSTM. We evaluate with accuracy, macro-averaged F1 per-family precision and recall and a confusion matrix.

Comparison models: To explain what the networks learn, we also trained two simpler classifiers. The first is a logistic regression on TF-IDF word features (unigrams, second on unigrams plus bigrams) with the regularization strength chosen on the validation set. The second is a logistic regression on 19 hand-built features describing length, formatting, and punctuation

Limitations: We used a single configuration and a single random seed with no hyperparameter search so small differences between the CNN and BiLSTM should not be taken as evidence one is 100% better then the other.

RQ1: Can we identify which LLM generated a response? Use LLM_output as input, conduct classification to detect which LLM generated it. Report the performance of your CNN&RNN performance

Model	Test accuracy
CNN	94.60%
BiLSTM	94.30%
Bag of words, unigrams (baseline)	86.30%
Bag of words, unigrams + bigrams (baseline)	89.70%

RQ1: per-family precision and recall on the test set (output only)						
Family	CNN precision	CNN recall	BiLSTM precision	BiLSTM recall		
Claude	93.30%	95.60%	92.60%	95.10%		
GPT	94.40%	95.60%	95.80%	94.00%		
Gemini	96.20%	92.50%	94.50%	93.60%		

```
CNN (output only) | test acc 0.9458 | macro-F1 0.9458
              precision    recall  f1-score   support

      Claude      0.933     0.956     0.945      1873
         GPT      0.944     0.956     0.950      1857
      Gemini      0.962     0.925     0.943      1807

    accuracy                          0.946      5537
   macro avg      0.946     0.946     0.946      5537
weighted avg      0.946     0.946     0.946      5537

confusion matrix (rows = true, cols = predicted):
          Claude   GPT  Gemini
Claude    1791    44      38
GPT         54  1775      28
Gemini      74    62    1671

BiLSTM (output only) | test acc 0.9426 | macro-F1 0.9426
              precision    recall  f1-score   support

      Claude      0.926     0.951     0.938      1873
         GPT      0.958     0.940     0.949      1857
      Gemini      0.945     0.936     0.940      1807

    accuracy                          0.943      5537
   macro avg      0.943     0.942     0.943      5537
weighted avg      0.943     0.943     0.943      5537

confusion matrix (rows = true, cols = predicted):
          Claude   GPT  Gemini
Claude    1782    35      56
GPT         68  1746      43
Gemini      75    41    1691
```

Our results show that both neural networks could identify the model family accurately from the response alone. On the test set of 5,537 responses, the CNN achieved 94.58% accuracy and a macro F1 of 0.9458, while the bidirectional LSTM achieved 94.26% accuracy and a macro F1 of 0.9426. Both of the results are well above the 33.3% accuracy you would expect from randomly choosing among the three families. The high macro F1 scores also show that performance was pretty strong across the classes, since this metric gives each family's F1 score equal weight. The results support fact that responses contain patterns associated with their source family, allowing the classifiers to identify Claude, GPT, and Gemini even when the user prompt is not provided.

The results per each family showed that this accuracy was not just one easily recognized class. For the CNN, recall was 95.6% for both Claude and GPT, and 92.5% for Gemini. Gemini had the highest precision, 96.2%. This means that predictions labeled Gemini were usually correct, although the CNN missed more actual Gemini responses than responses from the other families. The BiLSTM also performed consistently, with the F1 scores for each family that were 0.938 for Claude, 0.949 for GPT, and 0.940 for Gemini. The CNN's overall accuracy was only 0.32 percentage points higher than the BiLSTM so this single run doesn't say that CNNs are generally better for the task. Our finding is that both architectures successfully classified the three broad family labels in this dataset. Identifying exact model versions or responses from other datasets would require additional experiments.

RQ2: Does the user's prompt help identify the LLM? Compare classifiers using: Input only; Output only; Input + Output Questions you may consider:
i) Can the LLM be predicted from the user's prompt even before seeing its response?
ii) Does including the input improve classification?
iii) If input-only classification works surprisingly well, why might this happen?

RQ2: test accuracy by what the classifier sees (chance = 33.3%)

Input to the classifier	CNN	BiLSTM
Input (prompt) only	34.40%	32.30%
Output (response) only	94.60%	94.30%
Input + output	94.30%	92.50%

CNN | Input only | test acc 0.3435 | macro-F1 0.2689

	precision	recall	f1-score	support
Claude	0.343	0.658	0.451	1873
GPT	0.344	0.359	0.351	1857
Gemini	0.308	0.002	0.004	1807
accuracy			0.344	5537
macro avg	0.332	0.340	0.269	5537
weighted avg	0.332	0.344	0.272	5537

confusion matrix (rows = true, cols = predicted):

```
       Claude  GPT  Gemini
Claude    1232  638       3
GPT       1185  666       6
Gemini    1170  633       4

BiLSTM | Input only | test acc 0.3229 | macro-F1 0.3198
              precision    recall  f1-score   support

      Claude      0.329     0.407     0.364      1873
         GPT      0.322     0.258     0.287      1857
      Gemini      0.315     0.302     0.308      1807

    accuracy                          0.323      5537
   macro avg      0.322     0.322     0.320      5537
weighted avg      0.322     0.323     0.320      5537

confusion matrix (rows = true, cols = predicted):
       Claude  GPT  Gemini
Claude     762  509     602
GPT        790  480     587
Gemini     761  500     546

 Input + output
train/val/test sizes: {'train': 43859, 'val': 5486, 'test': 5537}
 CNN | Input + output | test acc 0.9429 | macro-F1 0.9430
              precision    recall  f1-score   support

      Claude      0.935     0.944     0.940      1873
         GPT      0.961     0.936     0.948      1857
      Gemini      0.933     0.949     0.941      1807

    accuracy                          0.943      5537
   macro avg      0.943     0.943     0.943      5537
weighted avg      0.943     0.943     0.943      5537

confusion matrix (rows = true, cols = predicted):
       Claude   GPT  Gemini
Claude    1768    40      65
GPT         60  1739      58
Gemini      62    31    1714

 BiLSTM | Input + output | test acc 0.9254 | macro-F1 0.9254
              precision    recall  f1-score   support

      Claude      0.907     0.942     0.924      1873
         GPT      0.948     0.922     0.935      1857
      Gemini      0.923     0.911     0.917      1807

    accuracy                          0.925      5537
   macro avg      0.926     0.925     0.925      5537
weighted avg      0.926     0.925     0.925      5537

confusion matrix (rows = true, cols = predicted):
       Claude   GPT  Gemini
Claude    1764    38      71
GPT         77  1713      67
Gemini     104    56    1647
```

To find out whether the user's prompt helps identify the source family, we compared models that received the prompt by itself, the response alone, or both together. All three settings used the same labeled example and the same training, validation, and test splits, so their results could be compared on the same exact test set. The input only models received up to 512 prompt tokens, and the output only models received up to 512 response tokens. For the combined setting, we placed up to 128 prompt tokens before a separator token, and then up to 512 response tokens. This keptthe response window used in RQ1. Each CNN and BiLSTM was trained separately for its input setting, using the same training procedure and selecting its best weights based on validation accuracy.

The prompt by itself did not provide reliable information about the model family. The input only CNN achieved 34.35% test accuracy, only a bit above the 33.3% chance baseline, while the input only BiLSTM achieved 32.29%. The macro F1 scores were 0.2689 and 0.3198 respectively. The CNN's accuracy is more important to look at alongside its confusion matrix. It generally favored Claude and rarely predicted Gemini, correctly identifying only 4 of the 1,807 Gemini examples. Therefore the slightly higher accuracy doesn't really indicate useful performance across all the classes. The BiLSTM made more predictions across the classes but still couldn't identify the source well enough. Compared with the roughly 94% accuracy that we got from responses alone, these results show that most of the identifying information available to these classifiers was in the generated response.

Including the prompt alongside the response did not improve overall performance much. The combined input CNN achieved 94.29% accuracy and a macro F1 of 0.9430, compared with 94.58% accuracy and 0.9458 macro F1 for the response only CNN. The combined input BiLSTM achieved 92.54% accuracy and a macro F1 of 0.9254, compared with 94.26% accuracy and 0.9426 macro F1 for its response only version. These represent accuracy drops of 0.29 and 1.72 percentage points. Because both settings retained up to 512 response tokens, the decrease can't be explained by the prompt taking space away from the response. One possible explanation we thought of is that adding text with little identifying information made it harder to iso thelate and find useful response patterns. Specifically in the longer sequence done by the BiLSTM. However, one random seed and configuration can't establish the cause of these differences, or how consistently they might happen.

The weak input only performance is consistent with how the dataset was collected. A user's prompt can be answered by different models, so the same request does not inherently identify one family which logically makes sense. We also grouped examples by prompt when creating the splits, this prevented an identical prompt from appearing in both training and testing. This ends up reducing the opportunity to predict a family by memorizing the same prompts. In another dataset, unusually high input only accuracy could reflect associations between prompts that were similar and the models chosen to answer them, or repeated prompts crossing the splits, rather than characteristics of generated writing. We did not observe that pattern here in this case. For our dataset and tested configurations, the prompt didn't give us very useful identifying information, and adding it to the response did not improve the classifiers' overall test performance either.

RQ4: What characteristics distinguish different LLMs?

To explain why the classifiers work we compared the 3 families on 19 measurable features (length, formatting, punctuation) trained Logistic regression classifiers on subsets of those features and on words alone and re created the neural models with certain signals removed.

Table 1: Share of responses using each formatting feature at least once

Feature	Claude	GPT	Gemini
Headers	19.90%	51.40%	28.50%
Bold	37.00%	71.90%	90.20%
Bullet points	46.20%	72.20%	78.60%
Numbered lists	31.60%	44.90%	55.30%
Table rows	1.60%	21.30%	6.30%
Horizontal lines	2.40%	49.90%	21.20%
Code blocks	19.20%	24.30%	20.50%
Emoji	3.40%	33.30%	3.70%
Em dashes	3.40%	46.50%	4.70%

Table 2 Median length of different characteristics

Median length	Claude	GPT	Gemini
chars	1205	2414	4332
words	170	330	611
lines	12	32	35
paragraphs	6	12	14

(i) What differences can we see across the models?

The three families differ clearly in both length and formatting. Claude writes the shortest responses on every measure with an median of 170 words, 12 lines and 6 paragraphs. GPT's responses are about twice as long (330 words, 32 lines, 12 paragraphs), and Gemini's are the longest at roughly 3.6 times Claude's length (611 words, 4,332 characters). GPT and Gemini

have similar median line and paragraph counts (32 vs. 35 lines, 12 vs. 14 paragraphs) even though Gemini writes about 1.85 times as many words which shows its lines or paragraphs are longer.

Formatting habits also separate the families (Table 1). Claude uses almost every formatting feature less often than the others. Only 1.6% of its responses contain a table row, 2.4% a horizontal line, 3.4% an emoji and 3.4% an m dash, which fits its short plain style. GPT has the highest rates for headers (51.4%), table rows (21.3%), horizontal lines (49.9%), emoji (33.3%), em dashes (46.5%) and code blocks (24.3%). Emoji and em dashes are basically exclusive to GPT since Claude and Gemini each stay under 5% on both. Gemini has the highest rates for bold (90.2%), bullet points (78.6%) and numbered lists (55.3%) so its long responses are heavy on emphasis and lists. It uses headers (28.5%) and horizontal lines (21.2%) much less than GPT. Code blocks appear at similar rates across the three (19.2% to 24.3%), so this feature does little to tell which model it is.

(ii) How would a feature only (formatting, punctuation etc) model perform?

Table 3. Logistic regression on the 19 hand-built style features (test set)

Features used	Count	Accuracy	Macro-F1
Length (characters, words, lines, paragraphs)	4	56%	0.56
Formatting (headers, bold, bullets, numbered lists, table rows, rules, code blocks)	7	66%	0.66
Punctuation (emoji, exclamation marks, question marks, em dashes, ends with a question)	5	58%	0.57
Sentence and lexical (word length, sentence length, MATTR)	3	52%	0.52
All features	19	**81%**	0.81
All features except length	15	77%	0.77

A model using only the 19 hand built style features reaches 81% test accuracy (macro-F1 0.81), way higher then chance (33.3%) but 13 to 14 points below the CNN (94.6%) and BiLSTM (94.3%) on raw text. Formatting is the strongest single group at 66%, followed by punctuation (58%), length (56%) and sentence and lexical features (52%). Combining all groups beats each one alone by at least 15 points so they capture different habits. Length is not essential since removing it lowers accuracy only from 81% to 77%. The gap to the neural models suggests they use information beyond these features like the specific wording of the responses.

(iii) Does removing certain signals substantially reduce performance?

Table 4. Neural models with signals removed (test accuracy)		
Setting	**CNN**	**BiLSTM**
Original responses (RQ1)	94.60%	94.30%
Formatting removed (markdown, emoji, dashes)	91.80%	91.60%
Length removed (≥256 tokens, all cut to 256)	95.80%	93.80%
Same window, length visible (reference)	94.10%	92.70%

Removing length did not hurt, accuracy stayed the same or rose, and the style-feature model lost only 4 points without it . Removing formatting cost both models 2.7-2.8 points. No single signal accounts for most of the accuracy because when one is removed the others still identify the family.

Limitations. The length-controlled set drops responses shorter than 256 tokens so it is not an exact comparison. Results come from a single random seed

(iiii) How much signal can we get from vocab alone?

Model	Test accuracy
CNN	94.60%
BiLSTM	94.30%
Bag of words, unigrams (baseline)	86.30%
Bag of words, unigrams + bigrams (baseline)	89.70%

Vocabulary alone carries a large share of the signal. A TF-IDF logistic regression that sees only which words appear with no word order or layout, reaches 86.3% test accuracy using single words and 89.7% with word pairs are added. Both are far above chance and above the 19 style features combined 81%. It is still 5 to 8 points below the CNN (94.6%) and BiLSTM (94.3%) so word order, phrasing and layout add some information beyond the vocabulary. The most informative words fit each family's language. Claude's includes words such as "would," "actually" and "typically." GPT's include "certainly," "sure," "below" and "summary." Gemini's include "okay," "breakdown" and "crucial."

Conclusion:

We learned from this project that LLM responses do have patterns that can help identify where they came from. Using responses alone, the CNN achieved 94.58% accuracy and the BiLSTM achieved 94.26%, showing that both architectures could distinguish the three family labels accurately in our dataset. Prompts alone performed near chance, and adding the prompt to the response also didn't improve either of the model's overall accuracy. The style and word experiments also told us that wording and formatting can provide useful information for identifying. Putting them together, the findings suggest that the generated response carries most of the information the one needs for this task, but the results do not establish that these patterns are reliable across other datasets or when models have updates.

The project also taught us that high accuracy is more meaningful when the data and evaluation are designed more carefully. Because we balanced the classes and kept identical prompts within one split, it helped make the comparisons more fair, while macro F1 and confusion matrices showed weaknesses that accuracy on its own could hide. Our experiments were limited to single turn responses and one random seed. In future work, we could repeat the experiments with several seeds include more model families and individual versions, and test longer responses. We would also investigate whether the classifiers work on unfamiliar task categories and newer model outputs. These tests would help determine whether the strong results reflect patterns that remain useful with this current dataset and others.

Citations:

LMArena. (n.d.). *arena-human-preference-140k* [Data set]. Hugging Face. https://huggingface.co/datasets/lmarena-ai/arena-human-preference-140k